

机器学习 $\approx$ 找到一个函数 f

狗-猫二分类 $f(\text{猫}) = \text{"猫"}$

元学习 $\approx$ 找到一个函数 F , 从而确定一个函数 f



图 15.6 元学习和机器学习的目标

务里面的训练数据输入到学习出来的学习算法里面, 得到一个分类器后, 再把测试数据输入进去, 得到分类的结果。我们把元学习里面的这个测试叫做跨任务测试, 因为它不是一般的测试。一般的机器学习, 我们的这个测试叫做单一任务测试, 因为我们是在一个任务里面进行测试。在元学习里面, 我们要测试的不是一个分类表现的好坏, 而是一个学习算法的表现的好坏, 所以在元学习里面为跨任务的测试。那有时候我们也在一些论文中会看到整个流程中一次单一任务的训练和一次跨任务的测试, 我们把这个流程叫做一个回合。所以在元学习里面, 我们是在一个回合里面进行训练和测试, 而在机器学习里面, 我们是在一个任务里面进行训练和测试。

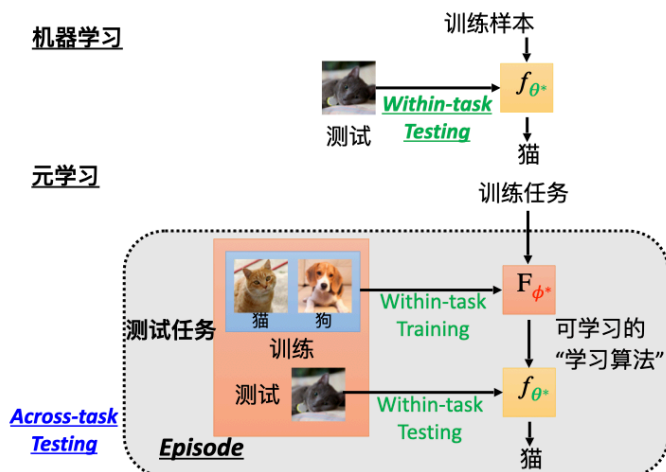


图 15.7 元学习和机器学习的框架对比

对于损失, 在机器学习中我们使用 $L(\theta) = \sum_{k=1}^K e_k$ 表示损失函数, 其中 e_k 表示第 k 个训练样本的损失, 其中的加和为所有训练数据在一个任务中的损失总和。在元学习里面, 我们使用 $L(\phi) = \sum_{n=1}^N l^n$ 表示损失函数, 其中的 l^n 表示第 n 个测试样本中的损失, 其中的加和为在所有任务中的损失总和。

对于训练的过程两者也有一些差异, 元学习的训练需要算 l^n , 也就是计算每一个小任务的损失函数, 在这个过程中, 我们需要做一次单一任务的训练 + 测试, 也就是一个回合。现

在假设我们的优化算法中, 要找一个 ϕ 让 $L(\phi)$ 最小, 那做这件事情的时候, 我们需要算这个 L 很多次, 也就是跨任务的训练包含了很多次的单一任务的训练 + 测试。这个是非常复杂的且耗时的。所以在文献中, 学习如何去初始化整个过程中, 往往把跨任务训练叫做外循环, 把单一任务训练叫做内循环。这是因为在跨任务训练里要跑好几次单一任务训练, 所以跨任务训练是外循环, 单一任务训练是内循环。不过, 外循环和内循环这些称呼通常只有在学习初始化的工作中才会有, 其他时候通常也不会这样叫。

刚才介绍的都是元学习和机器学习的差别, 他们其实也有很多的共同之处, 事实上很多在机器学习那边学到的知识和基本概念都可以直接搬到元学习来。举例来说, 在机器学习上面你会害怕训练数据上可能会有过拟合的问题, 那在元学习里面也有可能会有过拟合的问题, 比如机器学习到了一个学习算法, 这个学习算法在训练任务上做得很好, 面对一个新的测试的任务反而会做得不好, 所以元学习也有可能有过拟合的问题。如果遇到过拟合问题应该怎么办呢? 我们类比一下机器学习, 在机器学习里面, 最直观的方法就是收集更多的训练数据, 所以在元学习里面也可以做一样的事——收集更多的训练任务。也就是如果训练的任务越多, 就代表训练的数据样本越多, 那学习算法就越有机会可以泛化并用到新的任务上面。

另外, 我们在机器学习上会做数据增强, 也就是在训练的时候, 我们会把训练数据做一些变化, 比如说把图片做一些旋转、平移、缩放等等, 这样可以让训练数据更多。在元学习里面我们同样也可以做数据增强, 也可以想一些方法来增加训练任务, 比如说我们可以把训练任务做一些变化, 比如说把训练任务的类别做一些变化, 或者把训练任务的数据做一些变化等等。此外, 我们在做元学习的时候还是要做优化, 我们还是要想办法去找一个 ϕ 让 $l(\phi)$ 越小越好。假设我们最小化 $l(\phi)$ 的方法是梯度下降法, 那我们在做梯度下降法的时候, 我们还是要去调学习率, 只不过与机器学习不同, 我们需要调整的参数是可学习的学习算法的参数。有人可能会问, 既然都要调整参数, 那我何必还要用元学习, 直接讲每一个机器学习的问题调整参数不就可以了么? 其实并不是, 因为在元学习里面, 我们只需要把学习算法的参数调整好, 其就可以一劳永逸的使用到其他的任务当中, 而不需要每一个任务都去调整参数。这样的话, 我们就可以节省很多的时间, 也可以让我们的学习算法更加的有效率。

那既然说到要调参数就遇到另一个问题了, 在机器学习中我们不仅仅有训练样本和测试样本, 同时还有验证集的样本, 用验证集样本中的表现来选择你的模型。所以元学习中也应该要有用于验证的任务, 也就是说在元学习中, 我们应该要有训练任务、验证任务和测试任务。其中验证任务来决定训练学习算法的时候的一些超参数, 然后才跑在测试的任务中。

15.4 元学习的实例算法

前面我们已经讲完了元学习的基本概念, 接下来我们就要讲一些元学习的实例算法。在这里我们会介绍两个算法, 一个是**模型诊断元学习(model-agnostic meta-learning, MAML)**, 另一个是 **Reptile**。这两个算法都是在 2017 年提出来的, 而且都是基于梯度下降法进行优化的。那我们最常用的学习算法是梯度下降, 在梯度下降中, 我们要有一个网络架构, 同时初始化一下这个网络的参数 θ^0 。我们也有一些训练数据, 他们是从训练数据里面采样一个批次出来, 然后用这个批次来计算梯度, 并用这个梯度来更新参数, 从 θ^0 到 θ^1 , 接着再重新计算一次梯度, 再更新参数, 就反复这样下去, 直到次数够多, 输出一个满意的 θ^* 出来。

初始化的参数 θ^0 可以训练的, 一般 θ^0 是随机初始化的, 也就是从某一个固定的分布里面采样出来的。同时 θ^0 对结果往往有一定程度的影响, 好的初始化参数不好的初始化参数可以天差地远, 那我们能不能够透过一些训练的任务, 来找出一个对训练特别有帮助的初始化

参数呢? 这个方法是模型诊断元学习。

MAML 的基本思路是, 算法要最大化模型对超参数的敏感性。也就是说, 要让学习到的超参数让模型的损失函数因为样本的微小变化而有较大的优化。因此, 模型的超参数设置应该能够让损失函数变化的速度最快, 即损失函数此时有最大的梯度。因此, 损失函数被定义为每一个任务下该模型的损失函数的梯度的和。剩下要做的, 就是根据这个定义损失函数, 用梯度下降法进行求解。在训练的过程中算法会求取以下两次梯度。第一次求梯度: 针对每个任务, 计算损失函数的梯度, 进行梯度下降; 第二次求梯度: 对梯度下降后的参数求和, 再求梯度, 进行梯度下降。当然, MAML 有另外的变形就叫做 Reptile, 翻译过来叫爬虫, 大家可以自行了解。需要补充的是, 虽然在 MAML 中, 我们要去学习初始化参数的过程, 但是在其中我们也是有很多超参数需要自己决定的。

这里做一个联想, 我们在介绍自监督学习的时候, 我们也有提到好的初始化参数的重要性。在自监督学习中, 我们就是有很多的没有标记的数据, 可以用一些代理的任务去训练它, 比如说在 Bert 里面就是用填空题来训练模型, 在图像上也可以做自监督学习。比如把图片的其中一块盖起来, 让机器预测被盖起来的一块是什么东西, 机器就可以从中学到一些特征, 然后再把这些特征用在其他的任务上。当然, 现在在做图像的自监督学习的时候, 可能这个掩码的方法以及所谓的填空的方法不是最常用的, 目前比较流行用对比学习的方法。总之, 在自监督学习中我们会先拿一大些的数据去做预训练, 那预训练的结果我们也说它是好的初始化参数, 然后再把这些好的初始化参数用在测试的任务上。

那这 MAML 和自监督学习有什么不同呢? 其实它们的目的都是一样的, 都是要找到好的初始化参数, 但是它们的方法不一样。自监督学习是用一大些的数据去做预训练, 而 MAML 是用一大些的任务去做预训练。另外过去在自监督学习还没有兴起的时候, 也有一些方法是用一大些的任务去做预训练, 这个方法叫做多任务学习。具体来讲, 我们一样有好几个任务的数据, 并且把这些好几个任务的数据通通放在一起, 然后接下来我们同样可以找到一个好的初始化参数, 然后再把这个好的初始化参数用在测试的任务上, 这就是多任务学习。现在我们在做有关 MAML 研究的时候, 通常会把这种多任务学习的训练方法来当做元学习的基线。因为这两个方法他们用的数据都是一样的, 一边只是我们会把不同的任务分开, 另外一边把所有的任务的数据倒在一起。

其实 MAML 很像是域适应或者迁移学习, 也就是我们在某些任务上面学到的东西可以被迁移到另外一个域, 我们可以说他们是基于分类问题的域适应或者迁移学习。所以我们在研读文献的时候其实也不用太拘泥于这些词汇, 我们要真正要在意的是这些词汇背后所代表的含义是什么。

我们下面解释一下 MAML 的优势。首先有两个假设, 第一个假设是 MAML 找到的初始参数是一个很厉害的初始参数。它可以让我们的例如梯度下降这种学习算法快速找到每一个任务的参数。另外一个假设是这个初始化的参数它本来就和每一个任务上最终好的结果已经非常接近了, 所以我们直接应用很少几次的梯度下降就可以轻易的找到好的结果, 这个也是使得 MAML 有效的关键。当然 MAML 也有一些变形, 比如 ANIL (almost no inner loop)、First order MAML (FOMAML) 以及 Reptile 等等, 这里我们不做扩展。

除了可以学习初始化的参数外, MAML 还可以学习优化器, 如图 15.8 所示。我们在更新参数的时候, 需要决定比如说学习率、动量等等的参数。像学习率这种超参数进行自动更新的方法在很早以前就有了, NIPS2016 年就有一篇文章, 叫做“Learning to learn by gradient descent by gradient descent”。在这篇文章里面呢, 作者直接学习了优化器, 一般我们的优化

器都是人为规定的（比如 ADAM 等等），而这个文章中的参数是根据训练的任务自动学出来的。

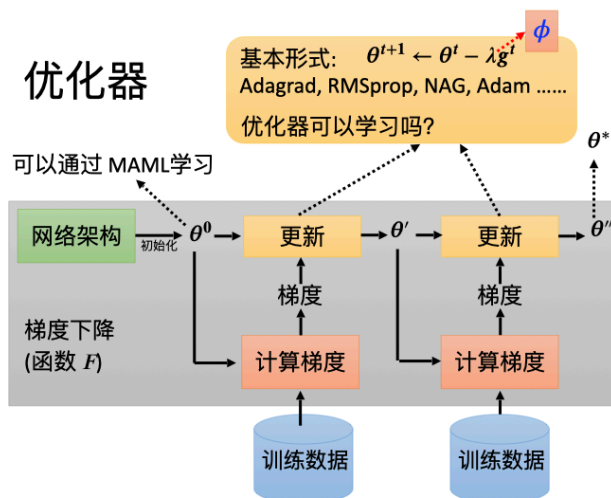


图 15.8 MAML 中可学习的优化器

当然我们还可以训练网络架构，这部分的研究被称为**神经网络架构搜索 (Neural Architecture Search, NAS)**。如果在元学习中我们学习的是网络的架构，讲网络架构当作 ϕ 的话，那我们就是在做 NAS。在 NAS 里面， ϕ 是网络架构，我们要找一个 ϕ 去最小化 $L(\phi)$ 。这其中我们做 $L(\phi)$ 的微分就有可能出问题了。当我们遇到优化问题时并且没办法上微分的时候，强化学习也许是一个解决方法。具体做法是，我们可以把 ϕ 想成是一个智能体的参数，这个智能体的输出是 NAS 中相关的超参数。举例来说，现在第一层的滤波器它的长是多少，宽是多少，步长是多少，数目是多少等等。我们智能体的输出就是 NAS 中相关的参数，然后接下来我们要训练智能体，让它去最大化一个回报，即 $-L(\phi)$ ，让他去最大化这个回报，就等于最小化 $L(\phi)$ ，也就是我们要找的最优的 ϕ 。

下面是从一篇文章中截取的一个 NAS 的例子，以此为例介绍 NAS 的过程，如图 15.9 所示。具体来讲，我们有一个智能体是 RNN 架构。这个 RNN 架构每次会输出一个网络架构有关的参数，比如它会输出滤波器的高是多少，再输出过滤器的宽是多少，接着再输出步长是多少等等。第一层第二层输出完了以后，接下来输出 $n+1$ 层，再输出 $n+2$ 层，以此类推。有了这些参数以后，就根据它们设计一个网络以后就去训练这个网络，这个过程其实就是之前我们介绍的单一任务训练。然后接下来就去做强化学习，我们可以把这这个网络他在测试数据上面的准确率当做回报来训练智能体，目标是最大化回报，这个过程其实就是跨任务训练。那除了强化学习以外，其实用演变算法也是可以的。本质上其实就是硬要把网络架构改一下，让它变得可以微分。其中有一个经典的做法叫做可微分架构搜索，这个方法的本质就是想办法让问题变的是可以微分，从而可以直接用梯度下降来最小化这个损失函数。

除了网络架构可以学习外，其实数据处理部分也有可能可以学习。我们在训练网络的时候，通常要做数据增强。那在元学习中我们可以让机器自动进行数据增强。另一个角度，我们在训练的时候，有时候会需要给不同样本不同的权重。具体操作的话就会有不同的策略，比如有的策略就是如果有一些样例距离分类边界线特别近，那说明其很难被分类，这样类似的样例也许就要给它们比较大的权重，这样网络就会聚焦在这些比较难分别的样例中，希望它们

网络架构搜索 (NAS)

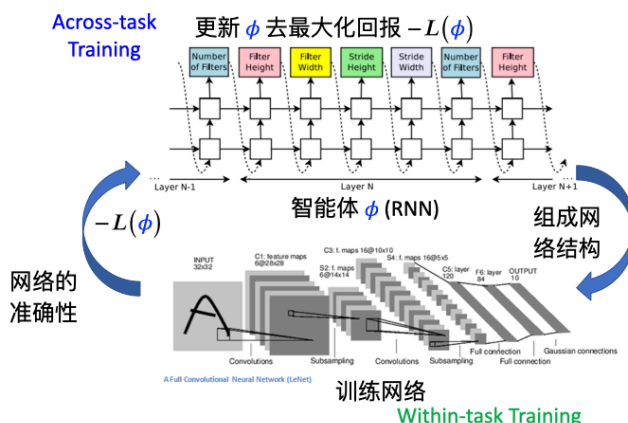


图 15.9 NAS 的实例

可以被学得比较好。但是也有文献有相反的结论，比如比较有干扰噪声的这些样本应该给它比较小的权重，这些样例如果比较接近分类边界线，可能代表它比较有噪声干扰，代表它可能标签本身就标错了，或者分类是不合理的等等，也许就应该给这些样本比较小的权重。那元学习中如何决定这个采样权重的策略呢？我们可以用学习的方式把采样策略直接学出来，然后根据采样数据的特性自动决定采样数据的权重如何设计。

到目前为止，我们看到的这些方法都是基于梯度下降再去做改进的，我们有没有可能完全舍弃掉梯度下降呢？比如，我们有没有可能直接学习一个网络，这个网络的参数 ϕ ，网络直接训练数据作为输入直接输出训练好的结果？如果真的有这样一个网络，它可以将训练数据作为输入输出训练好的网络的参数，那我们就可以让机器发明了新的学习算法。这个是有可能的，并且已经有了一些论文。不过到目前为止，我们还是把训练跟测试分成两个阶段，我们有一个学习算法使用训练数据进行训练，然后输出训练好的结果，并把训练好的结果用在测试数据上。我们想看看有没有可能更进一步，直接把整个回合，也就是一次训练加一次测试合并在一起。有一些工作，它就是直接把训练数据和测试数据当做网络的输入，网络输入完训练数据以后，机器也许学出了一个学习算法，也许找出一组参数等等，再它输入训练数据以后再给他测试数据，它就可以直接输出这些测试数据的答案。这个时候我们不再有训练和测试的分界，在一个回合里面不再分训练和测试，而是直接用个网络把训练和测试这件事情一次搞定。目前这种系列的方法叫做“learning to compare”，它又叫做基于度量的方法，这一系列的做法就可以看作是训练和测试没有分界，一个网络直接把训练数据和测试数据都读进去，而直接输出测试数据的结果。

15.5 元学习的应用

本章最后，我们再简单分享一些元学习的应用。现在在做元学习的时候，我们最常拿来测试元学习技术的任务叫做少样本图像分类，简单来讲就是每一个任务都只有几张图片，每一个类别只有几张图片。比如我们使用图 15.10 的案例为例说明。现在分类的任务是分为三个类别，每个类别都只有两张图片作为输入，我们希望通过这样一点点的数据就可以训练出一个模型。也就是给这个模型一张新的图片，它可以知道这张图片属于哪一个类别。在做这种

少样本图像分类的时候，我们会常看到一个名词叫做 N 类别 K 样例下的分类任务，这个名词是什么意思呢？ N 类别 K 样例的分类任务，它的意思就是在每一个任务里面，我们有 N 个类别，而每一个类别我们只有 K 个样例。举例来说，在图 15.10 这个例子里面我们有三个类别，每一个类别只有两个样例，那它就是 3 类别 2 样例分类。在元学习里，如果我们要教机器能够做 N 类别 K 样例分类，那意味着说我们需要准备很多的 N 类别 K 样例下的分类任务当做训练的任务，这样机器才能够学到 N 类别 K 样例的算法。

• 每一个类别只有几个图片



- N 类别 K 样本 分类: 在每一个任务重, 有 N 个类别, 每一个类别 K 个样本.
- 在元学习中, 你需要准备许多 N 类别 K 样本 任务作为训练和测试任务。

图 15.10 少样本的案例分析

那要怎么去找一系列的 N 类别 K 样例下的任务呢？在文章中最常见的一种做法是使用 Omniglot 当做基准，Omniglot 是一个手写的数据集，它有 1623 个不同的字符，每一个字符有 20 个样例。那有这些字符以后呢，我们就可以去制造 N 类别 K 样例下的分类。比如我们从 Omniglot 里面选出 20 个字符，然后每一个字符就只取一个样例，这样就得到一个 20 类别 1 样例的分类任务。如果我们把这个任务当做训练数据，那我们就可以让机器学习到 20 类别 1 样例的分类算法。如果我们把这个任务当做测试数据，那我们就可以测试机器在 20 类别 1 样例的分类任务上的表现。同理，我们可以制造出 20 类别 5 样例的分类任务，这个任务里面每一个类别都有 5 个样例，然后我们可以把这个任务当做训练数据，让机器学习到 20 类别 5 样例的分类算法。

在使用 Omniglot 的时候，我们会把字符分成两半，一半是拿来制造训练任务的字符，另外一半是拿来制造测试任务的字符。如果我们要去制造一个 N 类别 K 样例的任务，那么就是从这些训练任务的字符里面先随机采样 N 个字符，然后这 N 个字符每个字符再去采样 K 个样例，集合起来就得到一个训练的任务。对于测试的任务，就从这些测试的字符里面拿出 N 个字符，然后每个字符采样 K 个样例，你就得到一个 N 类别 K 样例下的测试任务。这样我们就可以把 Omniglot 当做一个基准，然后在这个基准上面测试不同的元学习算法。

总之，元学习不是只能用非常简单的任务，今天在学界已经开始把元学习推向更复杂的任务，我们也一直希望未来元学习这个技术能够真的用在现实的应用上，可以走得多远好。

第 16 章 终身学习

终身学习本质上基于人类对于人工智能的想象，期待人工智能可以像人类一样能够持续不断地学习。

16.1 灾难性遗忘

如图 16.1 所示，我们先训练机器做任务一语音识别，再教它做任务二图像识别，接着再教它做第三个任务翻译，这样一来它就一起学会了这三个任务。我们不断去教会机器学习新的技能，等它学会成百上千个技能之后，它就会变得越来越厉害，以至于人类无法企及，这就是我们所说的终身学习 (LifeLong Learning, LLL)。

终身学习也称为持续学习 (continous learning)、无止尽学习 (never-ending learning)、增量学习 (incremental learning)。

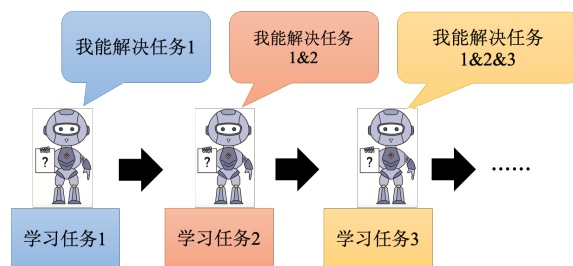


图 16.1 自监督学习框架

读者可能会疑惑，这个终身学习的目标过于远大，并且难以实现，其意义又在哪里呢？其实在真实的应用场景中，终身学习也是派得上用场的。举例来说，如图 16.2 所示，假设我们首先通过收集一些数据然后训练得到模型，模型上线之后它会收到来自使用者的反馈并且得到新的训练数据。这个时候我们希望能够形成一个循环，即模型上线之后我们得到新的数据，然后讲新的数据用于更新我们的模型，模型更新之后又可以收到新的反馈和数据，对应地再次更新我们的模型，如此循环往复下去，最终我们的模型会越来越厉害。我们可以把过去的任务看成是旧的数据，把新的数据即来自于反馈的数据，这种情景也可以看作是终身学习的问题。

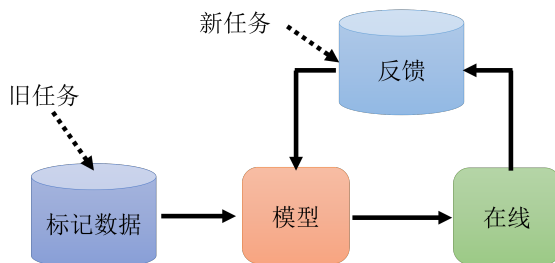


图 16.2 自监督学习框架

终身学习有什么样的难点呢？看上去不断更新它的数据和对应网络的参数就能实现终身

学习，但实际上没有那么容易。我们来看一个例子，如图 16.3 所示，假设我们现在有两个任务，第一个任务是手写数字识别，给一张包含噪声的图片，机器要识别出该图片中的数字“0”，第二个任务也是手写数字识别，只是给的图片噪声比较少，相对来说更容易。当然有读者可能会认为这不算两个任务，最多算同一个任务的不同域，这样说也没错。可能读者想象中的终身学习应该是先学语音识别然后图像识别之类的，但其实现在的终身学习都还没有做到那种程度。目前关于终身学习的论文中所说的不同任务大概指的就是本示例中同一个任务的不同域这种级别，只是我们把它当作不同的任务来对待。但即使是非常类似的任务，在做终身学习的过程中也会遇到一些问题，具体哪些问题我们接下来一一说明。

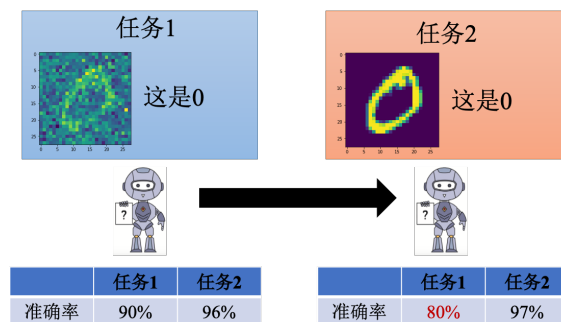


图 16.3 自监督学习框架

我们先训一个比较简单的网络来做第一个手写数字识别任务，然后再做任务二，任务一上的准确率是 90%，此时就算没有训过任务二，在任务二上也已经有了 96% 的准确率，可以说迁移得非常好，说明能够解决任务一，相应地也能解决任务二。那么接下来我们用同一个的模型，即在任务一中训练好的模型再去训练任务二，结果我们发现在任务二上的准确率变得更好了（97%）。但是比较糟糕的事情是此时机器已经忘了怎么去做任务一了，即在任务一上的准确率从 90% 降到了 80%。有读者可能会想是不是网络设置的太过简单然后导致出现这样的现象，但实际上我们把任务一和任务二的数据放在一起让这个网络同时去学的时候，会发现机器是能够同时学好这两个任务的，如图 16.4 所示。

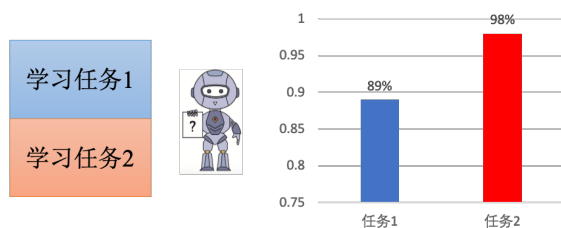


图 16.4 自监督学习框架

接下来举一个自然语言处理方面的例子，即 QA 的任务，QA 指的是给一篇文档，模型训练之后能够基于这个文档回答一些问题。当然为了简化，我们会将一个更简单的实例，即“bAbi”，它是一类非常早期和基础的研究任务，总共有二十个子任务。

如图 16.5 所示，其中第五个任务是给了三个句子，即 Mary 把蛋糕给了 Fred，Fred 把蛋糕给了 Bill，Bill 给了牛奶给 Jeff，最后问“谁给了蛋糕给 Fred？”类似于这种简单问题，其他任务包括第十五个任务也同理。我们的目标是让 AI 依次去学这二十个任务，要么让一个模型

Task 5: Three Argument Relations Mary gave the cake to Fred. Fred gave the cake to Bill. Jeff was given the milk by Bill. Who gave the cake to Fred? A: Mary Who did Fred give the cake to? A: Bill	Task 15: Basic Deduction Sheep are afraid of wolves. Cats are afraid of dogs. Mice are afraid of cats. Gertrude is a sheep. What is Gertrude afraid of? A: wolves
---	---

图 16.5 bAbi 任务示例

同时学这二十个任务，要么用二十个模型，每个模型分别学一个任务，这里我们主要讲的是前者。最后我们实验的结果如图 16.6 所示。

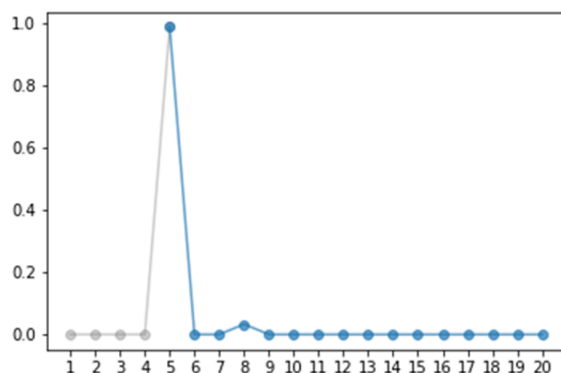


图 16.6 任务 5 的准确率（依次学习 20 个任务）

可以看到，一开始我们没有学过任务五，所以准确率是零。当开始学第五个任务之后，准确率达到百分百，然而开始学下一个任务的时候，准确率就开始暴跌，即学习新的任务之后一下子就完全忘了前面的任务。读者可能以为模型本身就没有学习那么多任务的能力，但其实不是的。如图 16.7 所示，当模型同时学二十个任务的时候，我们会发现机器是有潜力能够学习多个任务的，当然这里第十九个任务可能有点难，模型的准确率非常低。

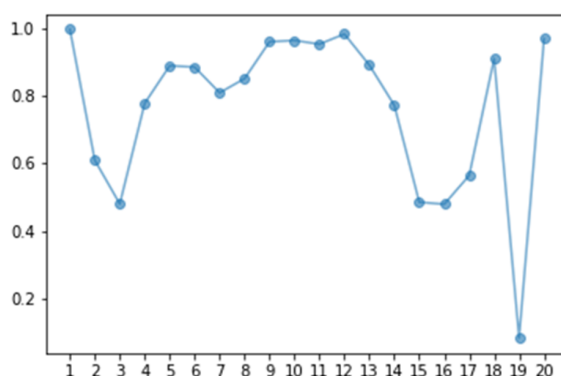


图 16.7 所有任务的准确率（同时学习 20 个任务）

当模型依次学习多个任务的时候，它好像一个上下两端都接有水龙头的池子，新的任务从上面水龙头进来，旧的东西就从下面水龙头那里冲掉了，永远学不会多个技能，这种情况称为**灾难性遗忘 (catastrophic forgetting)**。我们知道即使是人类也可能会有遗忘的时候，而这里在遗忘前面我们加上了灾难性这个形容词，意在强调模型的这个遗忘不是一般的遗忘，而是特别严重程度的遗忘。

讲到这里我们接下来就需要知道怎么去解决这个灾难性遗忘的问题了。在我们讨论具体

的技术之前,也许读者会有这样一个问题。刚才我们的例子有提到模型是能够同时学多个人任务的,这种学习方式称为**多任务学习 (multitask learning)**,既然有这个多任务学习的例子,那为什么还要去做终身学习的事情呢?但其实这种多任务学习会有这样一个问题,加上我们需要学习的任务不再是简简单单的二十个,而是一千个,那么我们在学第一千个任务的时候,按照这个逻辑还得把前面九百九十九个任务的数据放在一起训练,这样需要的时间是比较久的。

就好比假设一个人要学一门新的课程,那他就必须要把这辈子上过的所有课程都学过一遍才有办法学习新的任务,这样其实是比较低效的。而且随着要学习的任务越来越多,所需要的训练时间也会越来越长。但如果我们能够解决终身学习的技术,那么其实就能够高效地学习多种新任务了。当然这种多任务学习也并非没有意义,我们通常把多任务学习的效果当成终身学习的上界,即我们在做终身学习的时候往往会先跑一个多任务学习的结果,来帮助我们参考终身学习的上限在哪里。

这里可能读者又会有一个问题,即为什么我们不能每个任务都分别用一个模型呢?其实这样做会有一些问题,首先这样做可能会产生很多个模型,这样对机器存储也是一个挑战。另外不同任务之间可能是有共通的,从任务 A 学到的数据也可能在学习任务 B 的时候有所参考。类似的还有一个迁移学习的概念,虽然终身学习和迁移学习都是让模型同时学习多个任务,但它们的关注点是不一样的。在迁移学习中,我们在意的是模型在前一个任务学习到的东西能不能对第二个任务有所帮助,只在乎新的任务做得如何。而终身学习更注重的是在解决完第二个任务之后能不能再回头解决第一个任务。

16.2 终身学习评估方法

现在我们可以先看看怎么评判终身学习技术做不做得好的一些标准。在做终身学习之前,得先有一系列任务让模型去学习,其实通常都是比较简单的任务。如图 16.8 所示,任务一就是常规的手写数字识别,任务二其实还是手写数字识别,但只是把每一个数字用某一种特定的规则打乱,称之为排列,这种算是比较难的,还有更简单的就是把数字右转一次。

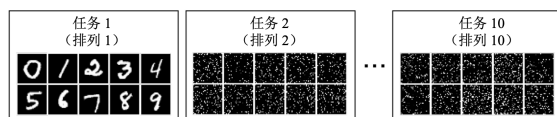


图 16.8 终身学习手写数字识别示例

具体的评估方式如图 16.9 所示,首先有一排任务,并且有一个随机初始化的参数,用在这 T 个任务上,得到对应的准确率。然后让模型先学第一个任务,然后在所有任务上分别测一次准确率,即 $R_{1,1}, \dots, R_{1,T}$,依此类推。直到学完所有的任务,得到一个准确率的表格,来评估终身学习的结果。

具体的最终准确率公式表示为

$$\frac{1}{T} \sum_{i=1}^T R_{T,i} \quad (16.1)$$

另一种评估方法叫做反向迁移,即

$$\frac{1}{T-1} \sum_{i=1}^{T-1} R_{T,i} - R_{T,i} \quad (16.2)$$

		测试			
		任务1	任务2		任务T
随机初始化		$R_{0,1}$	$R_{0,2}$		$R_{0,T}$
训练之后	Task 1	$R_{1,1}$	$R_{1,2}$		$R_{1,T}$
	Task 2	$R_{2,1}$	$R_{2,2}$		$R_{2,T}$
	⋮				
	Task T-1	$R_{T-1,1}$	$R_{T-1,2}$		$R_{T-1,T}$
	Task T	$R_{T,1}$	$R_{T,2}$		$R_{T,T}$

图 16.9 评估终身学习的准确率表格

16.3 终身学习的主要解法

解决终身学习问题，即主要解决灾难性遗忘的问题，通常来讲学术界有几种主要的研究思路。我们首先将第一种主要的解法，即选择性的突触可塑性 (selective synaptic plasticity)。顾名思义，就是只让神经网络中的某些神经元之间的连接具有可塑性，其余的必须被固化，这类方法又叫做基于正则的方法。我们可以先回顾一下会发生灾难性遗忘这种现象，例如现在在任务一和任务二，并且处于简化考虑，假设训练的模型只有两个参数 θ_1 和 θ_2 。如图 16.10 所示，分别表示了模型在不同任务上的损失函数，如果颜色越偏蓝色，代表损失越小，反之越大。

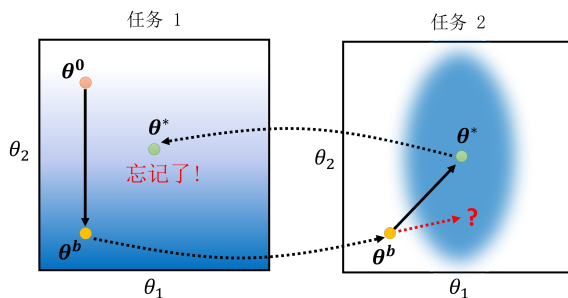


图 16.10 灾难性遗忘示意图

我们先让模型训练任务一，比如给一个随机化的初始参数 θ^0 ，用梯度下降的方法更新足够多次数的参数后得到 θ^b 。接下来继续训练任务二，我们先把 θ^b 复制过来放到任务二上，那么由于任务二的损失截面是不同的，即蓝色的区域不同，通过多次迭代我们可能会把参数更新到 θ^* 的位置。然后我们把在任务二上训练好的参数 θ^* 拿回到任务一上使用时，会发现并没有办法得到好的结果。因为这个 θ^* 只是在任务二上表现较好，而不见得会在任务一上有较低的损失和好的表现，这就是灾难性遗忘产生的原因。

那怎么解决这个问题呢？其实对于一个任务而言，要实现较低的损失是可能有很多种不同的参数组合的，比如在任务二中可能所有椭圆内的参数都能有好的表现，而对于任务一则是偏下方的位置都能实现较低的损失。那么如果在跑完任务一之后在训练任务二时，我们的参数不是像右上角移动形成 θ^* ，而是只往右移动，让最终得到的参数同时处于任务一和任务二较低的损失区域，这种情况下是有可能不会产生灾难性遗忘的。对于这种方式，基本的思想就是每一个参数对我们过去学过的任务的重要性程度是不同的，因此在学习新的任务时，我

们尽量不要动那些对过去任务很重要的参数，而是去学一些其他的对新任务比较重要的参数。假设 θ^b 是前一个任务学出来的参数，在选择性的突触可塑性这一解法中会给每一个参数 θ_i^b 赋予一个系数 b_i ，这个系数代表着对应的参数对过去的任务到底重不重要，也称作“守卫”。因此在更新参数的时候，会改写损失函数，如下：

$$L'(\theta) = L(\theta) + \lambda \sum_i b_i (\theta_i - \theta_i^b)^2$$

原来的损失函数写成 $L(\theta)$ ，在学习新任务时不会直接去最小化这个损失函数，否则就会发生灾难性遗忘的问题。我们会在右边一项增加一个当前参数 θ_i 与之前对应的参数 θ_i^b 的平均差并乘上我们讲的重要性系数 b_i 。这里 $b_i = 0$ 时，表示我们并不关心学习新任务时的参数需要跟过去的参数有什么联系，即一般的训练，这种情况下就容易发生遗忘。另一种极端就是当 b_i 趋近于无穷大时，即模型参数不肯在新任务上发生妥协，此时可能不会忘记旧的任务，但是也很有可能学不好新的任务，这种情况我们称作不妥协。

接下来比较关键的是 b_i 要怎么设定的问题，也就是某个参数 b_i 到底对任务的重要性有多少。其实有一种简单的控制变量的方法，就是移动或改变某个参数，如图 16.11 所示，我们移动 θ_1^b 时会发现在一定范围内损失值都是很小的，即接近最优的参数，那么我们就可以认为这个参数在一定范围可变，相应的重要性参数 b_1 就可以很小，即这个参数对旧任务来说不是很重要，反之像 θ_2^b 这种不能随意移动的参数对应的重要性参数就必须很大。

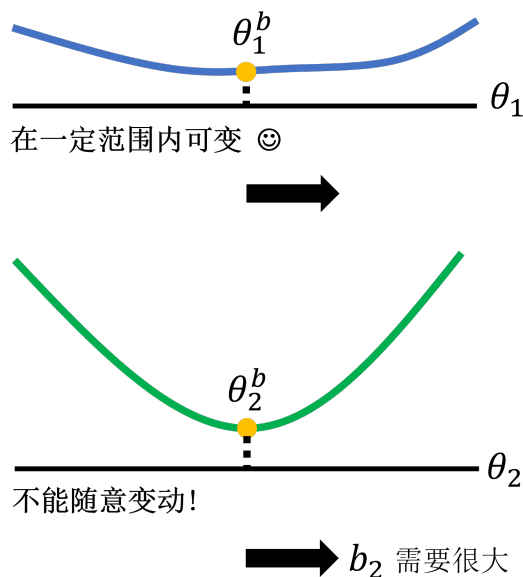


图 16.11 重要性参数设定示例

当然随着后续研究的深入， b_i 的设定也会有各种各样的方法，感兴趣的读者可以了解相关论文，这里就不一一讲解了。

其实在基于正则的方法出现之前，还有一类方法，叫做梯度回合记忆 (gradient episodic memory, MMD)，它不是在参数上做限制，而是在梯度更新的方向上做限制，因此又被称为基于梯度的方法。原理如图 16.12 所示，它在计算当前任务梯度方向的同时，也会回去算历史任务对应的梯度方向，然后把两个梯度进行向量求和，得出实际的梯度方向，这样持续更新就能尽可能接近一个不会陷入灾难性遗忘的最优解了。当然新的梯度方向需要满足大于等于

第 17 章 网络压缩

网络压缩 (network compression) 是一个很重要的方向, Bert 或 GPT 之类的模型很大, 能不能把这些硕大无朋的模型它缩小, 能不能够简化这些模型, 让它有比较少量的参数, 但是跟原来的性能其实是差不多的呢, 这就是网络压缩想要做的事情。很多时候, 需要把这些模型用在资源受限的环境下。有时候我们会需要把这些机器学习的模型。举个例子, 跑在智能手表, 这些边缘设备 (edge device) 只有比较少的内存, 只有比较少的计算力。模型如果太过巨大, 手表可能会是跑不动的, 所以需要比较小的模型。

Q: 为什么会需要在这些边缘设备上面跑模型呢? 为什么不把数据传到云端, 直接在云端上做计算, 再把结果传回到边缘设备?

A: 以手表为例, 为什么一定要在手表上面做计算呢? 一个常见的理由是延迟的问题。假设需要把数据传到云端, 云端计算完再传回来, 那中间就会有一个时间差。假设边缘设备是自驾车的一个传感器, 也许自驾车的传感器需要做非常即时的回应, 需要把数据传到云端再传回来, 中间的延迟太长了, 也许会长到是不能接受的。虽然在未来 5G 的时代延迟可能可以忽略不计, 但需要在边缘设备上面做计算的理由, 这个理由就是隐私。如果需要把数据传到云端, 云端的系统持有者不就看到我们的数据了。因此为了保障隐私, 也许在智能手表上直接进行计算并进行决策是一个可以保障隐私的做法。

接下来介绍五个以软件为导向的网络压缩技术, 这些技术只是在软件上面对网络进行压缩, 都不考虑硬件加速的部分。

17.1 网络剪枝

第一个技术是**网络剪枝 (network pruning)**。网络剪枝就是要把网络里面的一些参数剪掉。剪枝就是修剪的意思, 把网络里面的一些参数剪掉。为什么可以把网络里面的一些参数剪掉呢? 这么大的网络里面有很多很多的参数, 每一个参数不一定都有在做事。参数这么多的时候, 也许很多参数只是在划水, 什么事也没有做。这些没有做的参数放在那边, 就只是占空间而已, 浪费计算资源而已。为何就把它剪掉呢? 所以网络剪枝就是把一个大的网络中没有用的那些参数把它找出来, 把它扔掉。人刚出生的时候, 脑袋是空空的, 神经元跟神经元间没什么连接。在六岁的时候会长出非常多的连接。但是随着年龄渐长, 有一些连接就慢慢消失了。这个跟网络剪枝有异曲同工之妙。网络剪枝不是太新的概念, 早在这个 90 年代, Yann Le Cun 的论文“Optimal Brain Damage”是讲网络剪枝, 他把大脑 (网络) 剪枝, 把它剪掉一些权重看成是一种脑损伤, 最优的意思是要找出最好的剪枝的方法, 让一些权重被剪掉之后, 但是对这个脑的损伤是最小的。

网络剪枝其框架如图 17.1 所示, 首先, 先训练一个大的网络。接下来去衡量这个大的网络里面每一个参数或者是每一个神经元的重要性, 去评估一下有没有哪些参数是没在做事的, 或有没有哪些神经元是没在做事的。怎么评估某一个参数有没有在做事呢, 怎么评估某一个参数重不重要呢。最简单的方法也许就是看它的绝对值。如果这个参数的绝对值越大, 它可能越能对整个网络的影响越大。或者如果它的绝对值越接近零。也许对整个网络的影响越小, 也许对我们任务的影响越小。

我们可以评估每一个神经元的重要性, 把神经元当作修剪的单位。怎么看一个神经元重

不重要呢？比如计算这个神经元输出不为零的次数等等。总之有非常多的方法来判断一个参数或一个神经元是否重要。把不重要的神经元或是不重要的参数就剪掉，就把它从模型里面移出，就得到一个比较小的网络。但是做完这个修剪以后，通常模型的性能就会掉一点。因为有一些参数被拿掉了。所以这个网络当然是受到一些损伤，正确率就掉一点。但是我们会想办法让这个正确率再回升一点。可以把这个比较小的网络，把剩余没有被剪掉的参数，再重新做微调。把训练数据拿出来，把这个比较小的网络再重新训练一下。训练完之后，其实还可以重新再去评估一次每一个参数的正确性，还可以再去除掉，再剪掉更多的参数，再重新进行微调，这个步骤可以反复进行多次。

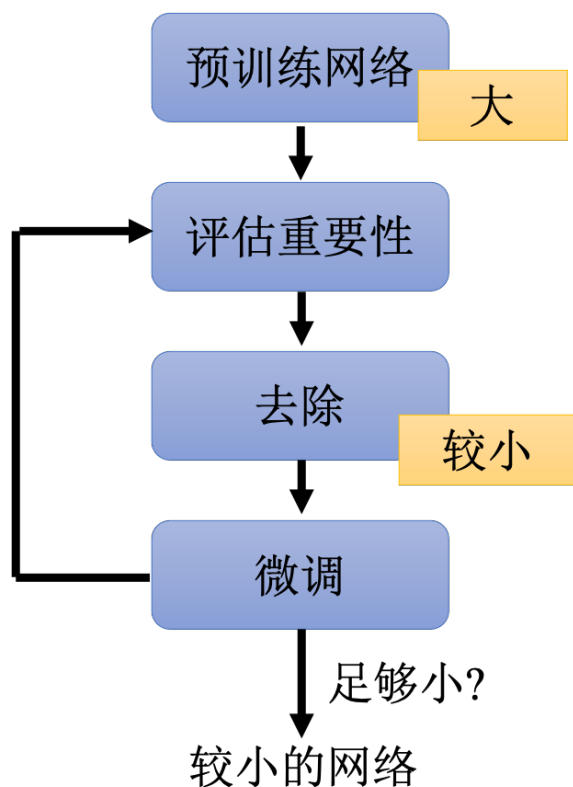


图 17.1 剪枝框架

为什么不一次剪掉大量的参数，因为在实验上，如果一次剪掉大量的参数，可能对网络的伤害太大了，可能会大到用微调也没有办法复原。所以一次先剪掉一点参数。比如说只剪掉 10% 的参数，然后再重新训练，然后再重新剪掉 10% 的参数，再重新训练，反复这一个过程。我们可以剪掉比较多的参数，当网络够小以后，整个过程就完成了，就得到一个比较小的网络。而且这个比较小的网络，也许它的正确率跟大的网络是没有太大的差别的。修剪的单位可以以参数为单位，也可以以神经元来当作单位，这两者作为单位，在实现上会有显著不同。如果我们现在是以参数当作单位会发生什么事，假设我们是要评估某一个参数要不要被去掉，某个参数对整个任务而言重不重要能不能够被去掉，我们把这个不重要的参数去掉以后，得到的网络的形状可能会是不规则的。如图 17.2 所示，所谓不规则的意思是说，举例来说，红色的这个神经元连到接下来三个绿色的神经元，但第二个红色的神经元只连到两个绿色的神经元。或这个红色的神经元的输入只有两个蓝色的神经元，而这个红色的神经元的输入有四个蓝色的神经元。如果把参数当作单位来进行修剪的话，修剪完以后的网络的形状会是不规

则的。

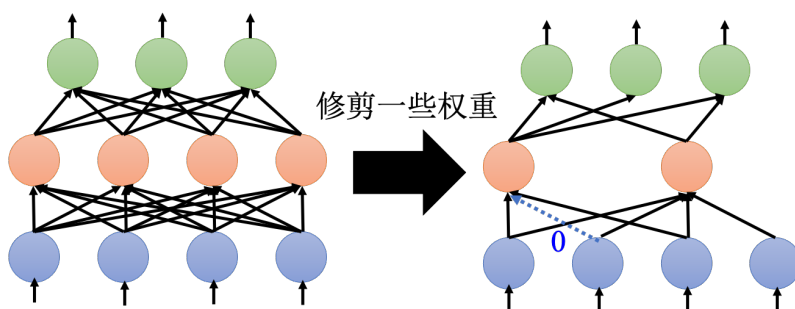


图 17.2 剪枝的问题

形状不规则最大的问题就是不好实现。用 PyTorch 要实现这种形状不规则的网络，不好实现。因为在 PyTorch 里面，定义第一个网络的时候，定义方法都是每一层有几个神经元，定义现在每一层要输入几个输入，输入有几个神经元，输出有几个神经元。或者输入多长的向量，输出有多长的向量。这种形状不固定的网络根本不好实现，而且就算硬是把这种形状不规则的网络实现出来，用 GPU 也很难加速。GPU 在加速的时候，就是把网络的计算看成一个矩阵的乘法，但是当网络是不规则的时候，不容易用矩阵的乘法来进行加速，我们不容易用 GPU 来进行加速，所以实际在做权重剪枝的时候，在实现上我们可能会把那些修剪掉的权重直接补零，就是修剪掉的权重不是不存在，它的值只是设为零。这样的好处是实现就比较容易，比较容易用 GPU 加速，但根本就没有真的把网络变小，虽然权重值是零，但还是存了这个参数，我们还是存了一个参数在内存里面，并没有真的把网络变小。这是以参数为单位来做剪枝的时候，实现上会遇到的问题。

如图 17.3 所示，紫色的这一条线是指稀疏程度 (sparsity)。稀疏程度就是有多少百分比的参数现在被修剪掉了。紫色的这条线的值都很接近 1，代表有接近大概 95% 以上的参数都被修剪掉了。网络剪枝其实是一个非常有效率的方法，往往可以修剪到 95% 以上的参数，但是准确率只掉 1 2% 而已。所以这边参数修剪时，有 95% 的参数都被丢掉了，只剩下 5% 的参数，这个网络变得很小，的计算要很快吧。但实际上我们发现根本就没有加速多少，甚至可以说根本就没有加速。图 17.3 中的长条图显示的是在三种不同的计算资源上面，加速的程度。加速的程度要大过 1 才有加速，加速程度小于 1 其实是变慢的。在多数情况下根本就没有加速，多数情况下其实都是变慢，也就是把一些权重修剪掉，结果网络形状变得不规则，真的用 GPU 加速的时候，反而没有办法真的加速它，所以权重剪枝不一定是一个特别有效的方法。

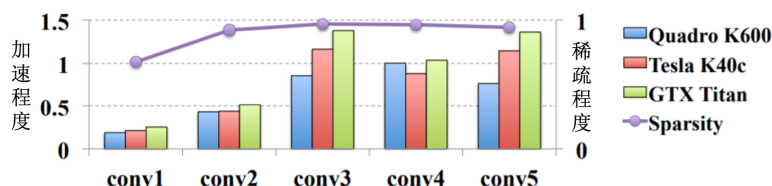


图 17.3 权重剪枝无法用 GPU 加速^[1]

神经元剪枝，即以神经元为单位来做剪枝也许是一个比较有效的方法。如果用神经元做单位来剪枝，丢掉一些神经元以后，网络的架构仍然是规则的。这种用 PyTorch 也比较好实

现,实现的时候,只要改那个每一个层输入、输出的那个维度就好了,也比较好用 GPU 来加速。

接下来要问一个问题:我们先训练一个大的网络,再把它变小,而且说小的网络跟大的网络的正确率没有差太多。不直接训练一个小的网络就好了,直接训练一个小的网络比较有效率,还训练大的网络变小干嘛,根本是舍本逐末,为什么不直接训练小的网络? 一个普遍的答案是大的网络比较好训练。如果直接训练一个小的网络,往往没有办法得到跟大的网络一样的正确率。我们可以先训练一个大的网络,再把它变小,正确率没有掉太多。但直接训练小的网络,得不到大的网络剪枝完变得小的网络一样的正确率。

为什么大的网络比较好训练? 有一个假说叫做彩票假说 (lottery ticket hypothesis)。假说代表说它不算是一个被实证的理论。但它现在只是一个假说而已,这个彩票假说是怎么解释为什么大的网络比较容易训练,直接训练一个小的网络没有办法得到跟大的网络一样的效果,一定要大的网络剪枝变小,结果才会好。彩票假说是这样说的,每次训练网络的结果不一定会一样。我们抽到一组好的初始的参数,就会得到好的结果;抽到一组坏初始的参数,就会得到坏的结果。但如何在彩票这个游戏里面得到比较高的中奖率,是不是就是包牌买比较多的彩券,可以增加中奖率,所以对一个大的网络来说也是一样的。大的网络可以视为是很多小的子网络 (sub-network) 的组合。如图 17.4 所示,我们可以想成是一个大的网络里面其实包含了很多小的网络。训练这个大的网络的时候,等于同时训练很多小的网络。每一个小的网络不一定可以成功的被训练出来。所谓成功的训练出来是说它不一定可以,通过梯度下降找到一个好的解,不一定可以训练出一个好的结果,不一定可以让它的损失变低。但是在众多的子网络里面,只要其中一个子网络成功,大的网络就成功了。而如果大的网络里面包含的小的网络越多,就好像是去买彩票的时候,买比较多的彩券一样,彩券越多,中奖的概率就越高。一个网络越大,它就越有可能成功的被训练起来。彩票假说在实验上是怎么被证实的? 它在实验上的证实方式跟网络的剪枝非常有关系,所以直接看一下在实验上是怎么证实彩票假说的。

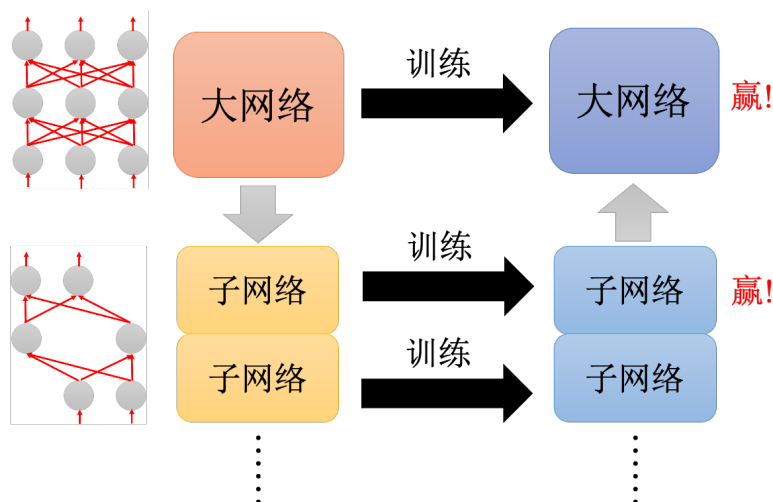


图 17.4 大网络包含了很多小网络

如图 17.5 所示,现在有一个大的网络,在这个大的网络上上面一开始的参数是随机初始化。把参数随机初始化以后,得到一组训练完的参数。训练完的参数用紫色来表示,接下来用网络剪枝的技术,把一些紫色的参数丢掉,而得到一个比较小的网络。如果直接把这个小的网络里

面的参数，再重新随机的去初始化，也就是重训练一个一样大小的小的网络，就是我们把这个网络复制一次。但是参数完全不一样，重新再训练一次。重新再训练一次，直接训练这个小的网络，训练不起来，训练一个大的再把它变小，没问题。但假设这一个小的网络，再重新初始化参数的时候，我们用的跟这组红色的参数是一模一样的，就训练得起来。

这两组参数虽然都是随机初始化的，但是这组绿色的参数跟这组红色的参数是没有关系的。而这边这些随机初始化的参数是直接从这个红色的参数里面选出对应的参数，就是这边有四个参数。我们就是把这边对应到的这四个参数直接把它复制过来，这边有四个参数。我们就把这里面对应到的四个参数直接复制过来，把这里的参数直接复制过来就训练了起来。如果用彩票假说来解释的话，就是这里面有很多子网络。而这一组初始化的参数，就是幸运的那一组可以训练得起来的子网络，所以用把这些网络，把这个大的网络训练完，再剪枝掉的时候，留下来的就是幸运的那些参数，可以训练得起来的那些参数，所以这一组初始化的参数是可以训练得起来的一个子网络。但是如果我们再重新随机初始化的话，就抽不到可以成功训练起来的参数，所以这个就是彩票假说。彩票假说非常地知名，它得到了 ICLR 2019 的最佳论文奖。

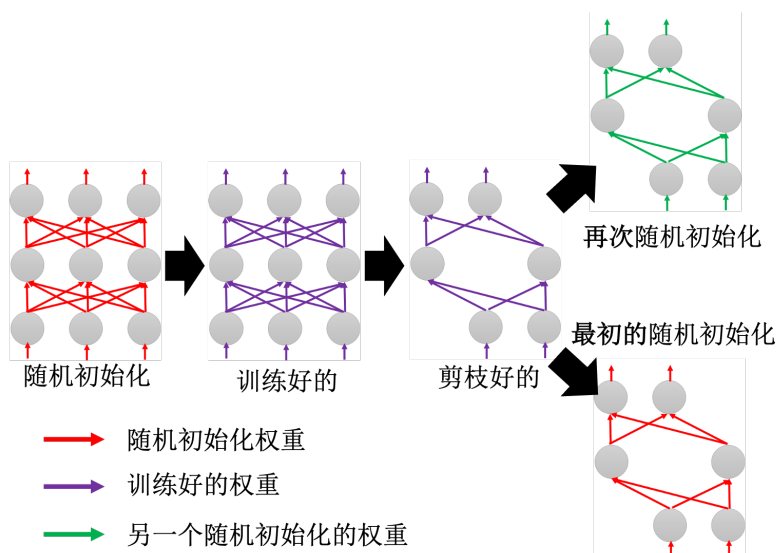


图 17.5 彩票假说

后面也有很多后续的研究，比如说有一篇有趣的研究叫做“Deconstructing Lottery Tickets: Zeros, Signs, and the Supermask”^[2]。解构这个彩票有什么有趣的结论，直接讲它的结论。第一个试了不同的剪枝策略，它做了一个非常完整的实验发现说，如果训练前跟训练后权重绝对值差距越大，剪枝那些网络得到的结果是越有效的。另外一个比较有趣的结果是，到底这一组好的初始化是好在哪里呢？如果我们只要不改变参数的正负号，就可以训练起来，小的网络只要不改变正负号，就可以训练起来。假设剪枝完以后，剩下的这个数。假设剪枝完以后，再把原来随机初始化的那些参数拿出来，它的值是 0.9, 3.1, -9.1, 8.5，可以完全不管它的数值，直接把正的数值的通通都用 $+\alpha$ 来取代，小于 0 的都用 $-\alpha$ 来取代，即 $+\alpha, +\alpha, -\alpha, +\alpha$ 。

用这组参数去初始化模型，这样也训练得起来，会跟用这组参数去初始化差不多。所以这个实验告诉我们说正负号是初始化参数，能不能够训练起来的关键，它的绝对值不重要，正负号才重要。

最后一个神奇的发现是，既然我们在想说一个大的网络里面有一些网络有一些子网络，它是特别是好的初始化的参数，它训练起来会特别地顺利。会不会一个大的网络里面甚至其实已经有一个子网络，它连训练都不用训练，直接拿出来就是一个好的网络呢？我们完全不用训练网络，直接把大的网络剪枝，就得到一个可以拿来做分类的分类器了，有没有可能是这个样子的呢。就好像米开朗基罗说：“塑像就在石头里，我只是把不需要的部分去掉”，会不会在整个大的网络里面，算参数都是随机的，其中已经有一组参数，它就已经可以做分类了，把多余的东西拿掉，直接就可以得到好的分类结果的。

但是彩票假说不一定是对的，论文“Rethinking the Value of Network Pruning”^[3] 是打脸彩票假说，而且神奇的是这篇文章跟彩票假说是同时出来的，它们同时出现在 ICLR 2019，它们得到了不太一样的结论。这篇文章试了两个数据集，还有好几种不同的模型，这个是没有剪枝过的网络的正确率，然后它试着去剪枝了一下网络，再重新去做微调。小的网络可以跟大的网络得到差不多的正确率。然后它说一般人的想像是，如果直接去训练小的网络，正确率会不如大的网络剪枝完以后的结果。

其实在这篇文章里面也有对彩票假说做出一些回应，它觉得彩票假说观察到的现象，也许只有在某一些特定的情况下才观察得到。根据这篇文章的实验，只有在学习率设比较小的时候，还有非结构化的（unstructured）的时候，就是剪枝的时候是以权重作为单位来做剪枝的时候，才能观察到彩票假说的现象，它发现说学习率调大，它就观察不到彩票假说的这个现象，所以到底彩票假说的正确性尚待更多的研究来证实。

17.2 知识蒸馏

接下来讲可以让网络变小的方法——**知识蒸馏 (knowledge distillation)**。先训练一个大的网络，这个大的网络在知识蒸馏里面称为教师网络 (teacher network)，其是老师。我们要训练的是真正想要的小的网络，即学生网络 (student network)。先训练一个大的网络称为教师网络。再根据这个大的网络来制造学生网络。在网络剪枝里面，直接把那个大的网络做一些修剪，把大的网络里面一些参数拿掉，就把它变成小的网络。在知识蒸馏里面是不一样的，这个小的网络（学生网络）是去根据教师网络来学习。假设要做手写数字识别，就把训练数据都丢到教师里面，教师就产生输出，因为这是一个分类的问题，所以教师的输出其实是一个分布。

比如教师的输出可能是看到这张图片 1 的分数是 0.7，7 的分数是 0.2，9 这个数字的分数是 0.1 等等。接下来给学生一模一样的图片，但是学生不是去看这个图片的正确答案来学习，它把老师的输出就当做正确答案，也就是老师输出 1 要 0.7，7 要 0.2，9 要 0.1。学生的输出也就要尽量去逼近老师的输出，尽量去逼近 1 是 0.7、7 是 0.2、9 是 0.1 这样的答案。学生就是根据老师的答案学，就算老师的答案是错的，学生就去学一个错的东西。

Q：为什么不直接训练一个小的网络？为什么不直接把小的网络去根据正确答案学习，而是要多加一个步骤先让大的网络学，再用小的网络去跟大的网络学习呢？这边的理由跟网络剪枝是一样的。

A：直接训练一个小的网络，往往结果就是没有从大的网络剪枝好。知识蒸馏的概念是一样的，因为直接训练一个小的网络，没有小的网络根据大的网络来学习结果要来得好。

其实知识蒸馏也不是新的技术，知识蒸馏最知名的一篇文章 Hinton 在 15 年的时候已经发表论文了。很多人会觉得知识蒸馏是 Hinton 提出来的，因为 Hinton 有一篇论文“Distilling the Knowledge in a Neural Network”。但其实在 Hinton 提出知识蒸馏这个概念之前，其实就有看过其他文章使用了一模一样的概念。举例来说，论文“Do Deep Nets Really Need to be Deep”是一篇 13 年的文章里面，也提出了网络蒸馏的想法。

为什么知识蒸馏会有帮助呢？一个比较直觉的解释是教师网络会提供学生网络额外的信息，如图 17.6 所示，如果直接跟学生网络这是 1，可能太难了。因为 1 可能跟其他的数字有点像，比如 1 跟 7 也有点像，1 跟 9 也长得有点像，所以对学生网络，我们告诉它：看到这张图片我们要输出 1。7、9 的分数都要是 0，可能很难，它可能学不起来，所以让它直接去跟老师学，老师会告诉它这是 1。我们没有办法让它 1 分，也没有关系。其实 1 跟 7 是有点像的，老师都分不出 1 跟 7 的差别。老师说 1 是 0.7，7 是 0.2，学生只要学到 1 是 0.7，7 是 0.2 就够了。这样反而可以让小的网络，学得比直接从头开始训练，直接根据正确的答案要学来得要好。

Hinton 论文里面甚至可以做到教师告诉学生哪些数字之间有什么样的关系这件事情，就可以让学生在完全没有看到某些数字的训练数据下，就可以把那个数字学会。假设训练数据里面完全没有数字 7，但是教师在学的时候有看过数字 7，但是学生从来没有看过数字 7。但光是凭着教师告诉学生说 1 跟 7 有点像，7 跟 9 有点像这样的信息，都有机会让学生可以学到 7 长什么样子。就算它在训练的时候，从来没有看过 7 的训练数据。这是知识蒸馏的基本概念。

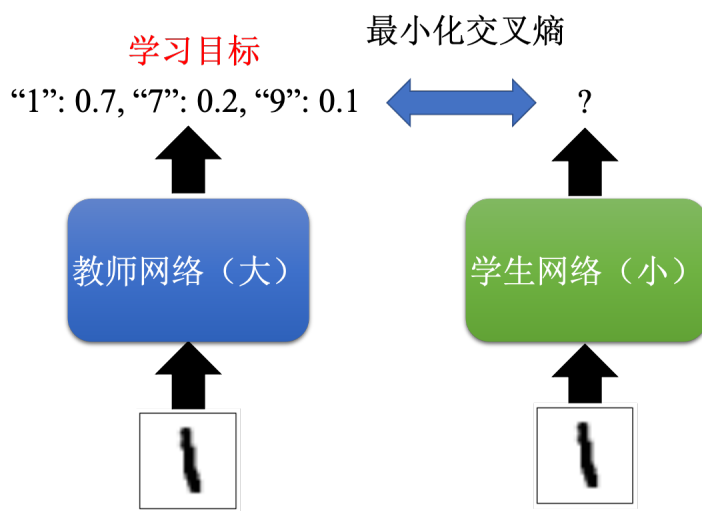


图 17.6 知识蒸馏

教师网络不一定要是单一的巨大网络，它甚至可以是多个网络的集成，训练多个模型，输出的结果就是多个模型，投票的结果就结束了。或者是把多个模型的输出平均起来的结果当做是最终的答案。虽然在比赛里面，常常会使用到集成的方法。如果在一个机器学习的比赛排行榜里面要名列前茅，往往凭借的就是集成技术，就是训练多个模型，把那么多的模型的结果通通平均起来。但是在实用上，集成会遇到的问题就训练了 1000 个模型，进来一笔数据，我们要 1000 个模型都跑过，再取它的平均，计算量也未免太大了。打比赛还勉强可以。要用在实际的系统上显然是不行的，可以把多个集成起来的网络综合起来变成一个，如图 17.7 所示。